ROS 2 Launch Files
& Parameters

Complete Tutorial

Applied to the Pathfinder Navigation System

What you will learn:

Launch Files — Start your entire system with one command instead of running each node in a separate terminal.

Parameters — Change node behaviour at runtime without editing or rebuilding code. Set from the terminal, from a launch file, or from a YAML config file.

Builds on top of:

`pathfinder_interfaces` `pathfinder_nodes` `navigate_server` `navigate_client`

No new packages. We add a `launch/` folder and a `config/` folder.

Environment

Item	Value
ROS 2	Humble Hawksbill
OS	Ubuntu 22.04 LTS
Python	3.10+
Build Tool	colcon
New files	3 launch files, 1 YAML config file

Contents

1 What Is a Launch File and Why Do We Need One? 3

2 What Are Parameters and Why Do We Need Them? 3

3 Package Structure 4

4 Create the Folders and Files 4

5 Update setup.py 5

6 Launch File 1 — Server Only 6
6.1 Every Part Explained . . . 7
6.2 Run it . . . 7

7 Launch File 2 — Service Server and Client Together 7
7.1 How Launch Arguments Work . . . 9
7.2 Run it . . . 9

8 Add Parameters to the Navigate Server 9
8.1 The Three Parameter Steps . . . 9
8.2 Updated __init__ method . . . 9
8.3 Updated is_valid method . . . 10
8.4 Updated sleep call . . . 10

9 Launch File 3 — Action Server and Client with Parameters 11
9.1 arguments vs parameters — The Difference . . . 12
9.2 Run it . . . 12

10 The YAML Config File 13
10.1 Write the YAML file . . . 13
10.2 Load YAML from a launch file . . . 13
10.3 Load YAML from the terminal (no launch file needed) . . . 14

11 Three Ways to Set Parameters — Summary 14
11.1 Way 1 — Terminal `-ros-args` . . . 14
11.2 Way 2 — Launch File `parameters=[{...}]` . . . 15
11.3 Way 3 — YAML File . . . 15
11.4 Comparison Table . . . 15

12 Build and Test Everything 16
12.1 Test 1 — Server only . . . 16
12.2 Test 2 — Service server and client with default coords . . . 16
12.3 Test 3 — Service server and client with custom coords . . . 16
12.4 Test 4 — Action server and client with default values . . . 16
12.5 Test 5 — Custom coordinates and slow speed . . . 16
12.6 Test 6 — Fast speed . . . 16
12.7 Test 7 — Show available arguments before running . . . 16

13 CLI Tools for Parameters at Runtime **17**

14 CLI Tools for Launch **17**

15 The Node Block — Every Field **18**

16 Complete File Flow **18**

17 Common Mistakes & Fixes **19**

18 Quick Reference **19**

18.1 Parameter Declaration in a Node 19

18.2 Commands Cheat Sheet 19

18.3 Summary Table 20

1 What Is a Launch File and Why Do We Need One?

Every time you have run the pathfinder system so far, you have needed at least two separate terminals. Terminal 1 runs the server. Terminal 2 runs the client. If your system grows to five nodes, you need five terminals, and you must remember the right order to start them, the right arguments to pass, and the right names to use. This does not scale.

A ROS 2 launch file solves this entirely. It is a Python script that describes your entire system — which nodes to start, what arguments to give them, what parameters to load, in what namespace to run them — and launches everything with a single command from a single terminal.

Launch files are not just a convenience. They are the standard way to deploy ROS 2 systems. Every real robot bring-up sequence, every Nav2 stack, every Gazebo simulation uses launch files. Learning them now means you will understand every ROS 2 project you encounter going forward.

```
Before launch files:
  Terminal 1: ros2 run pathfinder_nodes navigate_server
  Terminal 2: ros2 run pathfinder_nodes navigate_client 1 1 8 6
  Terminal 3: ros2 run pathfinder_nodes pathfinder_server
  ...

After launch files:
  ros2 launch pathfinder_nodes navigate.launch.py start_x:=1 start_y:=1
```

2 What Are Parameters and Why Do We Need Them?

Inside your nodes right now, values like grid size and step delay are hardcoded:

```
# navigate_server.py
COLS       = 10      <- hardcoded
ROWS       = 10      <- hardcoded
STEP_DELAY = 0.5     <- hardcoded
```

To change them you must open the file, edit it, and run `colcon build`. That is unacceptable in a real system. Robots in the field cannot be recompiled every time an operator wants to change a speed limit.

ROS 2 Parameters are the solution. You **declare** the parameter inside the node with a default value, and then **set** it from outside — from the terminal, from a launch file, or from a YAML configuration file — at runtime, with no code changes and no rebuild required.

```
# In the node -- declare once:
self.declare_parameter('step_delay', 0.5)
self.step_delay = self.get_parameter('step_delay').value

# Override from outside -- no rebuild needed:
ros2 run pathfinder_nodes navigate_server --ros-args -p step_delay:=2.0
ros2 launch pathfinder_nodes navigate.launch.py step_delay:=0.1
```

3 Package Structure

We add two new folders to the existing `pathfinder_nodes` package. No new packages are needed.

```
pathfinder_nodes/
 config/                          <- NEW: parameter config files
    navigate_params.yaml         <- NEW: YAML parameter values

 launch/                          <- NEW: launch files
    server.launch.py             <- NEW: launch server only
    pathfinder.launch.py         <- NEW: service server + client
    navigate.launch.py           <- NEW: action server + client

 pathfinder_nodes/
    __init__.py
    pathfinder_server.py
    pathfinder_client.py
    navigate_server.py           <- updated: uses parameters
    navigate_client.py

 package.xml
 setup.cfg
 setup.py                          <- updated: registers launch + config
```

4 Create the Folders and Files

```
cd ~/ros2_ws/src/pathfinder_nodes

# Create launch folder and files
mkdir launch
touch launch/server.launch.py
touch launch/pathfinder.launch.py
touch launch/navigate.launch.py

# Create config folder and YAML file
mkdir config
touch config/navigate_params.yaml
```

Critical: File Extension

Launch files MUST end in `.launch.py`. ROS 2 only recognises this exact pattern. A file named `navigate.py` will NOT be found by `ros2 launch`.

5 Update setup.py

ROS 2 must know about your new folders so it can install them. Update `setup.py` to register both `launch/` and `config/`:

Listing 1: setup.py — updated with launch and config registration

```python
import os
from glob import glob
from setuptools import find_packages, setup

package_name = 'pathfinder_nodes'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),

        # Register all launch files
        (os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py')),

        # Register all config/YAML files
        (os.path.join('share', package_name, 'config'),
            glob('config/*.yaml')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='you@email.com',
    description='Pathfinder nodes with launch files and parameters',
    license='Apache-2.0',
    extras_require={'test': ['pytest']},
    entry_points={
        'console_scripts': [
            'pathfinder_server = pathfinder_nodes.pathfinder_server:main',
            'pathfinder_client = pathfinder_nodes.pathfinder_client:main',
            'navigate_server   = pathfinder_nodes.navigate_server:main',
```

```
            'navigate_client    = pathfinder_nodes.navigate_client
                :main',
        ],
    },
)
```

Why glob?

`glob('launch/*.launch.py')` automatically finds all `.launch.py` files in the folder. This means any new launch file you create is registered automatically on the next build — you never need to update `setup.py` again for launch files.

6 Launch File 1 — Server Only

The simplest launch file. Starts just the action server node.

Listing 2: launch/server.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([

        Node(
            package    = 'pathfinder_nodes',
            executable = 'navigate_server',
            name       = 'navigate_server',
            output     = 'screen',
        )

    ])
```

6.1 Every Part Explained

Code	Purpose
`from launch import LaunchDescription`	Container that holds all launch actions
`from launch_ros.actions import Node`	Imports the Node action for starting ROS 2 nodes
`def generate_launch_description():`	ROS 2 calls this exact function name automatically
`return LaunchDescription([...])`	Returns the list of everything to launch
`package = 'pathfinder_nodes'`	Which ROS 2 package the node lives in
`executable = 'navigate_server'`	The entry point name from setup.py
`name = 'navigate_server'`	Name shown in ros2 node list
`output = 'screen'`	Print all node logs to the terminal

6.2 Run it

```
ros2 launch pathfinder_nodes server.launch.py
```

Expected Output

```
[INFO] [launch]: All log files can be found below /home/...
[INFO] [navigate_server-1]: process started with pid [...]
[navigate_server-1] [INFO] [navigate_server]: Navigate Action Server is ready.
[navigate_server-1] [INFO] [navigate_server]: Grid: 10 cols x 10 rows
```

7 Launch File 2 — Service Server and Client Together

Starts two nodes simultaneously. The server starts first, then the client.

Listing 3: launch/pathfinder.launch.py

```python
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration


def generate_launch_description():

    # Declare command line arguments with default values
    start_x_arg = DeclareLaunchArgument('start_x', default_value='1')
    start_y_arg = DeclareLaunchArgument('start_y', default_value='1')
```

```
    end_x_arg   = DeclareLaunchArgument('end_x',   default_value=
        '8')
    end_y_arg   = DeclareLaunchArgument('end_y',   default_value=
        '6')

    # Service server node
    server_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'pathfinder_server',
        name       = 'pathfinder_server',
        output     = 'screen',
    )

    # Service client node -- reads coordinates from launch
        arguments
    client_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'pathfinder_client',
        name       = 'pathfinder_client',
        output     = 'screen',
        arguments  = [
            LaunchConfiguration('start_x'),
            LaunchConfiguration('start_y'),
            LaunchConfiguration('end_x'),
            LaunchConfiguration('end_y'),
        ],
    )

    return LaunchDescription([
        start_x_arg,
        start_y_arg,
        end_x_arg,
        end_y_arg,
        server_node,
        client_node,
    ])
```

7.1 How Launch Arguments Work

```
DeclareLaunchArgument('start_x', default_value='1')
        |
        |  declares: "this launch file accepts a 'start_x' argument"
        |            "if not provided, use '1' as default"
        v
LaunchConfiguration('start_x')
        |
        |  reads:    "whatever value 'start_x' currently has"
        |            (either the default or what was passed on CLI)
        v
Node(..., arguments=[LaunchConfiguration('start_x'), ...])
        |
        |  passes:   the value into sys.argv of the node process
        v
navigate_client.py
        |
        start_x = int(sys.argv[1])   <- receives the value
```

7.2 Run it

```
# With default coordinates (1,1) to (8,6)
ros2 launch pathfinder_nodes pathfinder.launch.py

# With custom coordinates
ros2 launch pathfinder_nodes pathfinder.launch.py \
  start_x:=0 start_y:=0 end_x:=9 end_y:=9
```

8 Add Parameters to the Navigate Server

Before writing the third launch file, update `navigate_server.py` to use parameters instead of hardcoded constants.

8.1 The Three Parameter Steps

Step	Code	Purpose
1. Declare	`self.declare_parameter('cols', 10)`	Register with default
2. Read	`self.get_parameter('cols').value`	Get current value
3. Use	`self.cols` everywhere	Replace hardcoded constant

8.2 Updated ___init___ method

Replace the class constants and `__init__` with:

Listing 4: navigate_server.py — updated __init__

```
def __init__(self):
    super().__init__('navigate_server')

    # Step 1: Declare parameters with default values
    self.declare_parameter('cols',       10)
    self.declare_parameter('rows',       10)
    self.declare_parameter('step_delay', 0.5)

    # Step 2: Read parameter values into instance variables
    self.cols       = self.get_parameter('cols').value
    self.rows       = self.get_parameter('rows').value
    self.step_delay = self.get_parameter('step_delay').value

    self.action_server = ActionServer(
        self,
        NavigatePath,
        'navigate_path',
        self.execute_callback
    )

    self.get_logger().info('Navigate Action Server is ready.')
    self.get_logger().info(
        f'Grid    : {self.cols} cols x {self.rows} rows'
    )
    self.get_logger().info(
        f'Delay   : {self.step_delay}s per step'
    )
```

8.3 Updated is_valid method

Replace the hardcoded `COLS` and `ROWS`:

Listing 5: navigate_server.py — updated is_valid

```
def is_valid(self, x, y):
    # Step 3: Use self.cols and self.rows instead of class
        constants
    in_bounds = 0 <= x < self.cols and 0 <= y < self.rows
    free      = (x, y) not in self.OBSTACLES
    return in_bounds and free
```

8.4 Updated sleep call

Listing 6: In execute_callback — replace STEP_DELAY

```
# Before:
time.sleep(self.STEP_DELAY)    # hardcoded class constant

# After:
time.sleep(self.step_delay)    # runtime parameter value
```

9 Launch File 3 — Action Server and Client with Parameters

The most complete launch file. Starts the action server with parameters and the action client with coordinates.

Listing 7: launch/navigate.launch.py

```python
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration


def generate_launch_description():

    # Navigation coordinate arguments
    start_x_arg = DeclareLaunchArgument('start_x', default_value='1')
    start_y_arg = DeclareLaunchArgument('start_y', default_value='1')
    end_x_arg   = DeclareLaunchArgument('end_x',   default_value='8')
    end_y_arg   = DeclareLaunchArgument('end_y',   default_value='6')

    # Server parameter arguments
    cols_arg       = DeclareLaunchArgument('cols', default_value='10')
    rows_arg       = DeclareLaunchArgument('rows', default_value='10')
    step_delay_arg = DeclareLaunchArgument('step_delay', default_value='0.5')

    # Action server node -- parameters passed inline
    server_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'navigate_server',
        name       = 'navigate_server',
        output     = 'screen',
        parameters = [{
            'cols':       LaunchConfiguration('cols'),
            'rows':       LaunchConfiguration('rows'),
            'step_delay': LaunchConfiguration('step_delay'),
        }],
    )

    # Action client node -- coordinates passed as arguments
    client_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'navigate_client',
        name       = 'navigate_client',
```

```python
        output      = 'screen',
        arguments   = [
            LaunchConfiguration('start_x'),
            LaunchConfiguration('start_y'),
            LaunchConfiguration('end_x'),
            LaunchConfiguration('end_y'),
        ],
    )

    return LaunchDescription([
        start_x_arg,
        start_y_arg,
        end_x_arg,
        end_y_arg,
        cols_arg,
        rows_arg,
        step_delay_arg,
        server_node,
        client_node,
    ])
```

9.1 arguments vs parameters — The Difference

	arguments	parameters
What it is	Values passed as `sys.argv` to the process	ROS 2 parameter system values
Read in node as	`int(sys.argv[1])`	`self.get_parameter('name').value`
Set in launch as	`arguments=[LaunchConfiguratio`	`parameters=[{'key': value}]`
Used for	Coordinates, filenames, mode strings	Node configuration: speed, size, thresholds
Can change at runtime	No — set at startup only	Yes — `ros2 param set /node name value`

9.2 Run it

```bash
# Default values
ros2 launch pathfinder_nodes navigate.launch.py

# Custom coordinates and parameters
ros2 launch pathfinder_nodes navigate.launch.py \
  start_x:=0 start_y:=0 end_x:=9 end_y:=9 \
  cols:=10 rows:=10 step_delay:=1.0

# Fast navigation
ros2 launch pathfinder_nodes navigate.launch.py \
  start_x:=1 start_y:=1 end_x:=8 end_y:=6 \
  step_delay:=0.1
```

Expected Output with step_delay:=1.0

```
[navigate_server-1] [INFO] Grid    : 10 cols x 10 rows
[navigate_server-1] [INFO] Delay   : 1.0s per step
[navigate_client-2] [INFO] Action server connected!
[navigate_client-2]   Step  0 | START | Pos: (1,1) | Remaining: 14
[navigate_client-2]   Step  1 | UP    | Pos: (1,0) | Remaining: 13
...  (one line per second)
```

10 The YAML Config File

For systems with many parameters, a YAML file is cleaner than listing them all in a launch file. The YAML file lives in the `config/` folder and is loaded by name.

10.1 Write the YAML file

Listing 8: config/navigate_params.yaml

```yaml
navigate_server:
  ros__parameters:
    cols: 10
    rows: 10
    step_delay: 0.5
```

YAML Indentation is Critical

YAML is whitespace-sensitive. Use exactly 2 spaces for each indentation level. Never use tabs. The node name (`navigate_server`) must match the `name` field in your launch file exactly.

10.2 Load YAML from a launch file

Listing 9: Loading YAML in a launch file

```python
import os
from ament_index_python.packages import
    get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration


def generate_launch_description():

    # Resolve the full path to the YAML file
    params_file = os.path.join(
        get_package_share_directory('pathfinder_nodes'),
        'config',
        'navigate_params.yaml'
```

```
    )

    start_x_arg = DeclareLaunchArgument('start_x', default_value=
        '1')
    start_y_arg = DeclareLaunchArgument('start_y', default_value=
        '1')
    end_x_arg   = DeclareLaunchArgument('end_x',   default_value=
        '8')
    end_y_arg   = DeclareLaunchArgument('end_y',   default_value=
        '6')

    server_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'navigate_server',
        name       = 'navigate_server',
        output     = 'screen',
        parameters = [params_file],    # load entire YAML file
    )

    client_node = Node(
        package    = 'pathfinder_nodes',
        executable = 'navigate_client',
        name       = 'navigate_client',
        output     = 'screen',
        arguments  = [
            LaunchConfiguration('start_x'),
            LaunchConfiguration('start_y'),
            LaunchConfiguration('end_x'),
            LaunchConfiguration('end_y'),
        ],
    )

    return LaunchDescription([
        start_x_arg, start_y_arg,
        end_x_arg,   end_y_arg,
        server_node, client_node,
    ])
```

10.3 Load YAML from the terminal (no launch file needed)

```
ros2 run pathfinder_nodes navigate_server \
  --ros-args \
  --params-file ~/ros2_ws/src/pathfinder_nodes/config/
     navigate_params.yaml
```

11 Three Ways to Set Parameters — Summary

11.1 Way 1 — Terminal `-ros-args`

```
ros2 run pathfinder_nodes navigate_server \
  --ros-args \
  -p cols:=15 \
  -p rows:=15 \
  -p step_delay:=2.0
```

When to use

Quick testing. You want to try a different value immediately without changing any files.

11.2 Way 2 — Launch File `parameters=[{...}]`

```
Node(
    ...
    parameters = [{
        'cols':       LaunchConfiguration('cols'),
        'rows':       LaunchConfiguration('rows'),
        'step_delay': LaunchConfiguration('step_delay'),
    }],
)
```

When to use

Repeatable runs with known values. You want the same configuration every time you run the launch file, with the ability to override from CLI.

11.3 Way 3 — YAML File

```
navigate_server:
  ros__parameters:
    cols: 10
    rows: 10
    step_delay: 0.5
```

When to use

Production systems or nodes with many parameters. All configuration lives in one file that can be version controlled and swapped without touching launch files or node code.

11.4 Comparison Table

Method	Where	Best for	Rebuild?
`-ros-args -p`	Terminal	Quick tests	No
`parameters=[{...}]`	Launch file	Repeatable runs	No
`parameters=[yaml_file]`	Launch file	Many params, production	No
`-params-file`	Terminal	Loading YAML without launch	No

12 Build and Test Everything

```
cd ~/ros2_ws
colcon build --packages-select pathfinder_nodes
source install/setup.bash
```

12.1 Test 1 — Server only

```
ros2 launch pathfinder_nodes server.launch.py
```

12.2 Test 2 — Service server and client with default coords

```
ros2 launch pathfinder_nodes pathfinder.launch.py
```

12.3 Test 3 — Service server and client with custom coords

```
ros2 launch pathfinder_nodes pathfinder.launch.py \
  start_x:=0 start_y:=0 end_x:=9 end_y:=9
```

12.4 Test 4 — Action server and client with default values

```
ros2 launch pathfinder_nodes navigate.launch.py
```

12.5 Test 5 — Custom coordinates and slow speed

```
ros2 launch pathfinder_nodes navigate.launch.py \
  start_x:=0 start_y:=0 end_x:=9 end_y:=9 \
  step_delay:=1.0
```

12.6 Test 6 — Fast speed

```
ros2 launch pathfinder_nodes navigate.launch.py \
  start_x:=1 start_y:=1 end_x:=8 end_y:=6 \
  step_delay:=0.1
```

12.7 Test 7 — Show available arguments before running

```
ros2 launch pathfinder_nodes navigate.launch.py --show-args
```

Expected Output

```
Arguments (pass arguments as '<name>:=<value>'):
  start_x    : 1
  start_y    : 1
  end_x      : 8
  end_y      : 6
```

```
    cols       : 10
    rows       : 10
    step_delay : 0.5
```

13 CLI Tools for Parameters at Runtime

While the server is running, open a second terminal and use these commands:

```
# List all parameters of the running node
ros2 param list /navigate_server

# Get a specific parameter value
ros2 param get /navigate_server step_delay
ros2 param get /navigate_server cols

# Change a parameter while the node is running
ros2 param set /navigate_server step_delay 2.0

# Dump all current parameters to terminal
ros2 param dump /navigate_server

# Save all parameters to a YAML file
ros2 param dump /navigate_server > my_saved_params.yaml
```

Note on Runtime Changes

`ros2 param set` changes the parameter value in the running node. However, since our node reads parameters only in `__init__`, you must **restart the node** for the new value to take effect. Nodes that call `get_parameter()` inside callbacks will pick up changes immediately without restarting.

14 CLI Tools for Launch

```
# Show all arguments a launch file accepts
ros2 launch pathfinder_nodes navigate.launch.py --show-args

# List all nodes running after launch
ros2 node list

# List all topics
ros2 topic list

# List all services
ros2 service list

# List all actions
ros2 action list
```

15 The Node Block — Every Field

Listing 10: All available Node() fields

```
Node(
    package     = 'pathfinder_nodes',      # ROS 2 package name
    executable = 'navigate_server',        # entry point from setup
        .py
    name        = 'navigate_server',       # name in the ROS 2
        graph
    output      = 'screen',                # print logs to terminal
    arguments   = ['1', '1', '8', '6'],    # sys.argv arguments
    parameters = [{'step_delay': 0.5}],   # ROS 2 parameters dict
    parameters = ['/path/to/file.yaml'], # or: load from YAML
    remappings = [('/old_topic', '/new_topic')],  # rename topics
    namespace   = 'robot_1',               # prefix for all names
    env         = {'MY_VAR': 'value'},     # environment variables
)
```

16 Complete File Flow

```
ros2 launch pathfinder_nodes navigate.launch.py step_delay:=1.0
          |
          v
navigate.launch.py
  params_file = ".../config/navigate_params.yaml"
  DeclareLaunchArgument('step_delay', default='0.5')
  LaunchConfiguration('step_delay') = '1.0'   <- from CLI
          |
          |--- server_node
          |      parameters=[{'step_delay': '1.0'}]
          |      |
          |      v
          |    navigate_server.py (__init__)
          |      self.declare_parameter('step_delay', 0.5)
          |      self.step_delay = self.get_parameter('step_delay').value
          |      self.step_delay = 1.0   <- from launch file
          |
          |--- client_node
                 arguments=['1','1','8','6']
                 |
                 v
               navigate_client.py (main)
                 start_x = int(sys.argv[1]) = 1
                 start_y = int(sys.argv[2]) = 1
                 end_x   = int(sys.argv[3]) = 8
                 end_y   = int(sys.argv[4]) = 6
```

17 Common Mistakes & Fixes

Mistake	Error	Fix
File named `navigate.py` not `navigate.launch.py`	`unrecognized argument`	Always end launch files with `.launch.py`
Wrong `-show-args` position	`unrecognized argument`	Put after the file name: `file.launch.py -show-args`
Forgot to add `launch/` to `data_files` in `setup.py`	Launch file not found	Add `glob('launch/*.launch.py')` to `data_files`
Forgot to rebuild after editing `setup.py`	Old version runs	`colcon build` then `source install/setup.bash`
YAML node name does not match `name` in launch file	Parameters not loaded	Node name in YAML must exactly match `name=` in Node()
Using `parameters=[]` with hardcoded values instead of `LaunchConfiguration`	CLI arguments ignored	Wrap values with `LaunchConfiguration('key')`
Forgot `import os` and `from glob import glob` in `setup.py`	`NameError: os not defined`	Add both imports at the top of `setup.py`
YAML uses tabs instead of spaces	YAML parse error	Use 2 spaces per indent level, never tabs

18 Quick Reference

18.1 Parameter Declaration in a Node

```python
# In __init__:
self.declare_parameter('name', default_value)
self.name = self.get_parameter('name').value

# Types supported:
self.declare_parameter('count',  10)        # int
self.declare_parameter('speed',  0.5)       # float
self.declare_parameter('mode',   'fast')    # string
self.declare_parameter('debug',  False)     # bool
```

18.2 Commands Cheat Sheet

```bash
# Build and source
colcon build --packages-select pathfinder_nodes
source install/setup.bash
```

```
# Show launch arguments
ros2 launch pathfinder_nodes navigate.launch.py --show-args

# Run with default args
ros2 launch pathfinder_nodes navigate.launch.py

# Run with custom args
ros2 launch pathfinder_nodes navigate.launch.py \
  start_x:=0 start_y:=0 end_x:=9 end_y:=9 step_delay:=1.0

# Parameter tools
ros2 param list /navigate_server
ros2 param get  /navigate_server step_delay
ros2 param set  /navigate_server step_delay 2.0
ros2 param dump /navigate_server
```

18.3 Summary Table

What	File	Key Line
Register launch files	`setup.py`	`glob('launch/*.launch.py')`
Register config files	`setup.py`	`glob('config/*.yaml')`
Declare a parameter	node `.py`	`self.declare_parameter(...)`
Read a parameter	node `.py`	`self.get_parameter(...).value`
Declare CLI argument	launch file	`DeclareLaunchArgument(...)`
Read CLI argument	launch file	`LaunchConfiguration(...)`
Pass to node params	launch file	`parameters=[{key: value}]`
Pass to node args	launch file	`arguments=[LaunchConfiguration(...)]`
Load YAML	launch file	`parameters=[params_file]`

End of Tutorial — ROS 2 Launch Files & Parameters